

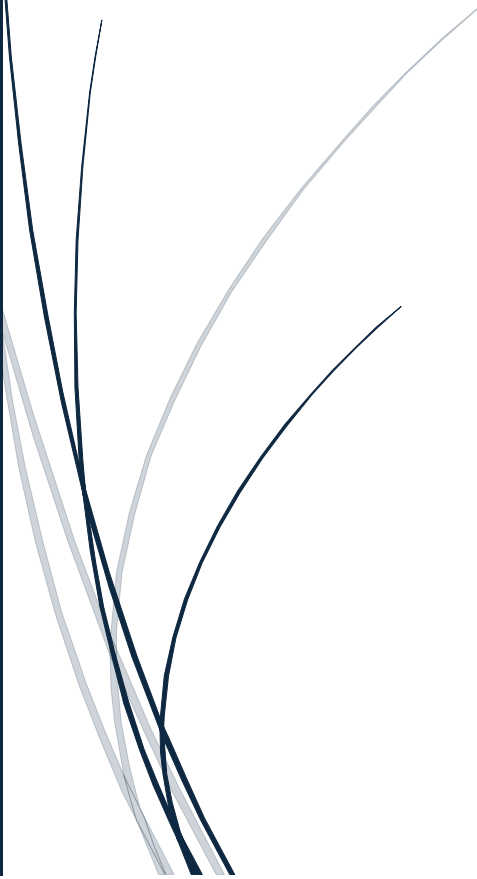


22 de septiembre de 2026

LAB: Traversal sequences blocked with absolute path bypass

CNO IV – Seguridad Informática

Profesor: Servando López Contreras
Universidad Politécnica de San Luis Potosí



Luis Felipe Jaramillo Barajas
MATRICULA: 183559

Contenido

1. Introducción	2
2. Objetivos	2
Objetivo general.....	2
Objetivos específicos	2
3. Preparación del laboratorio	3
4. Fase 1: Interceptación de la petición de imagen	3
5. Fase 2: Confirmación del filtro de traversal.....	5
6. Fase 3: Explotación — bypass mediante ruta absoluta	5
6.1 Explicación del payload.....	6
6.2 Resolución del laboratorio	7
7. Mitigación recomendada	8
8. Conclusiones	8

1. Introducción

Este informe documenta el proceso completo de una práctica de laboratorio realizada sobre la plataforma PortSwigger Web Security Academy, específicamente el laboratorio “File path traversal, traversal sequences blocked with absolute path bypass”. A diferencia del caso simple abordado previamente, en este laboratorio la aplicación implementa un filtro que elimina las secuencias de traversal (“../”) del parámetro de nombre de archivo antes de utilizarlo, por lo que la explotación directa mediante retrocesos de directorio ya no es posible.

El objetivo de este ejercicio fue identificar una vía alterna de explotación que evadiera dicho filtro, aprovechando que la aplicación trata el valor recibido como una ruta relativa a un directorio de trabajo por defecto, lo cual permite un bypass mediante el uso de una ruta absoluta, con el fin de acceder nuevamente al archivo `/etc/passwd` del servidor.

Todo el ejercicio se realizó dentro del entorno controlado que proporciona la propia plataforma de PortSwigger, utilizando Burp Suite Community Edition como proxy de interceptación para analizar y modificar las peticiones HTTP entre el navegador y el servidor de la aplicación vulnerable.

2. Objetivos

Objetivo general

Demostrar la capacidad de identificar y evadir un mecanismo de filtrado de secuencias de path traversal, explotando el comportamiento de resolución de rutas absolutas del sistema de archivos del servidor para acceder a un archivo fuera del directorio previsto por la aplicación.

Objetivos específicos

- Configurar Burp Suite Community Edition como proxy de interceptación entre el navegador y la aplicación objetivo.
- Interceptar y modificar una petición HTTP legítima para construir un payload de ruta absoluta.
- Acceder al contenido del archivo `/etc/passwd` evadiendo el filtro de la aplicación.
- Comprender los fundamentos teóricos del bypass mediante ruta absoluta.
- Proponer medidas de mitigación adecuadas para prevenir este tipo de evasión.

3. Preparación del laboratorio

El laboratorio utilizado, “File path traversal, traversal sequences blocked with absolute path bypass”, es proporcionado través de su plataforma Web Security Academy. Contiene una vulnerabilidad de path traversal en la funcionalidad que muestra las imágenes de los productos de la tienda en línea ficticia “We Like To Shop”. La aplicación bloquea las secuencias de traversal, pero trata el nombre de archivo proporcionado como una ruta relativa a un directorio de trabajo por defecto. El objetivo es recuperar el contenido del archivo `/etc/passwd`.



Figura 1. Descripción del laboratorio “Traversal sequences blocked with absolute path bypass” en PortSwigger Web Security Academy.

Al igual que en la práctica anterior, se utilizó Burp Suite Community Edition como herramienta de interceptación, con un proyecto temporal en memoria y configuración por defecto.

Concepto teórico: filtrado de secuencias de traversal. Una contramedida habitual contra path traversal consiste en detectar y eliminar del valor recibido las subcadenas “../” (o su variante con barra invertida “.\”) antes de construir la ruta final. Sin embargo, esta defensa solo neutraliza el vector de retroceso de directorios; no evita que la aplicación reciba y procese una ruta ya absoluta, lo cual constituye una vía de evasión distinta que debe contemplarse por separado.

4. Fase 1: Interceptación de la petición de imagen

Con el proxy de Burp Suite activo, se accedió a la tienda “We Like To Shop” correspondiente a esta instancia del laboratorio y se navegó por su catálogo de productos. En el historial de peticiones (HTTP history) se identificó nuevamente el patrón de solicitudes GET utilizado para cargar cada imagen:

```
GET /image?filename=37.jpg HTTP/2
```

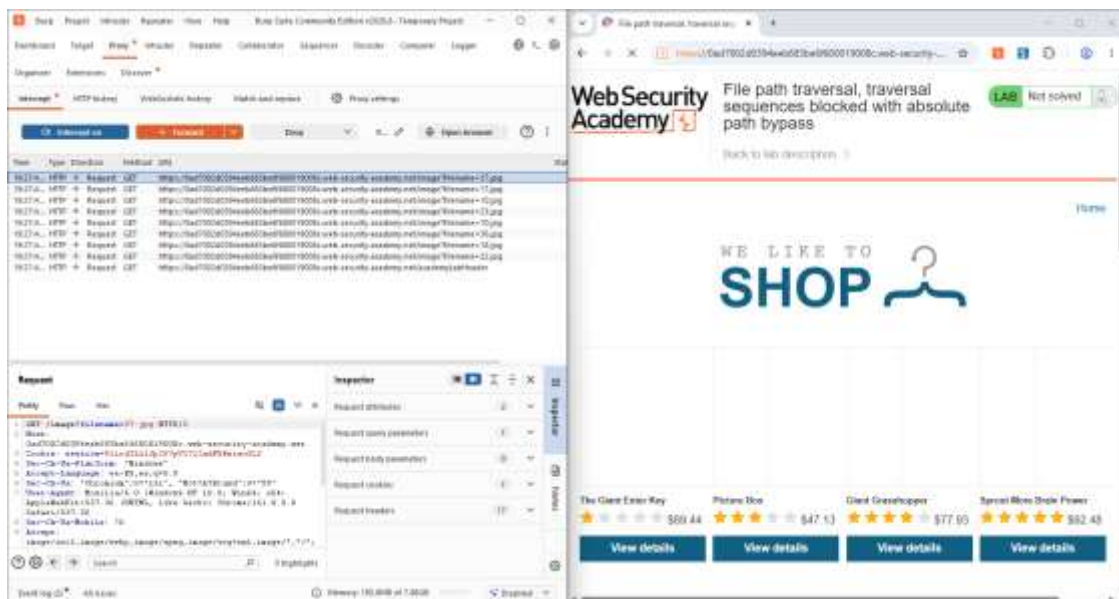


Figura 2. Historial de peticiones en Burp Proxy mostrando solicitudes GET /image?filename=N.jpg sobre la tienda “We Like To Shop”.

Se seleccionó una de estas peticiones y, mediante el menú contextual, se envió a la herramienta Repeater para poder modificar libremente el valor del parámetro filename.

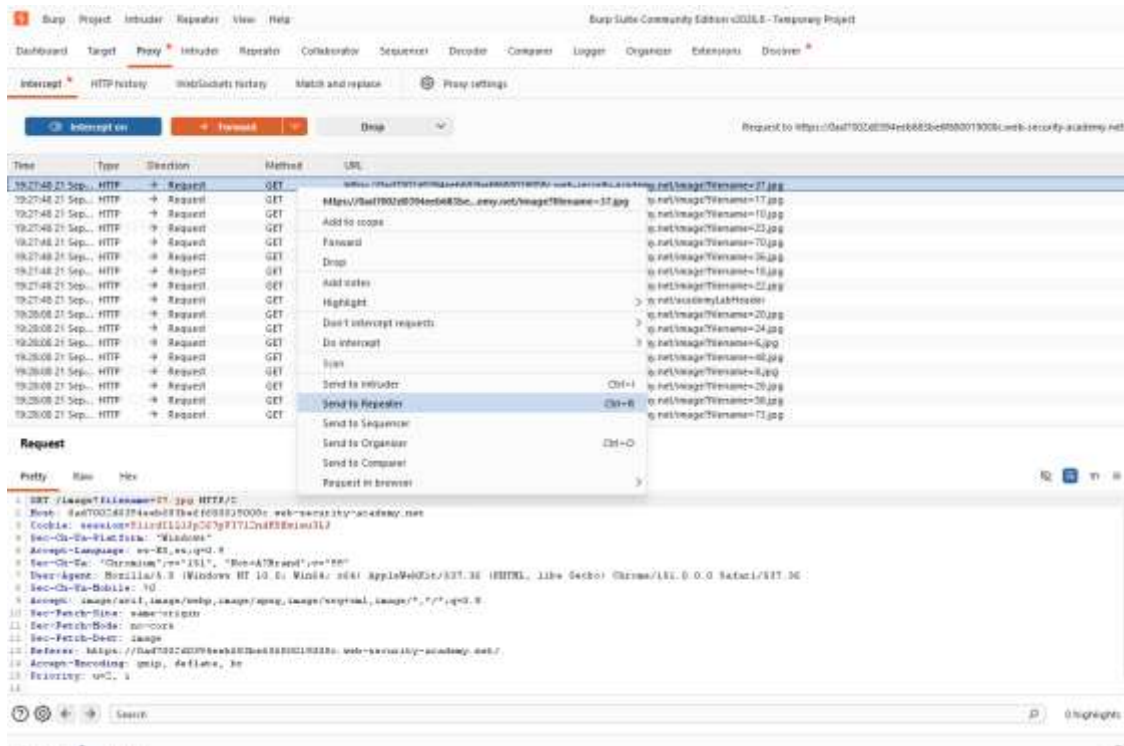


Figura 3. Envío de la petición seleccionada (filename=37.jpg) a Burp Repeater mediante “Send to Repeater”.

5. Fase 2: Confirmación del filtro de traversal

Dentro de Repeater se visualizó la petición original con el parámetro filename=37.jpg, confirmando el mismo punto de entrada identificado en el laboratorio anterior.

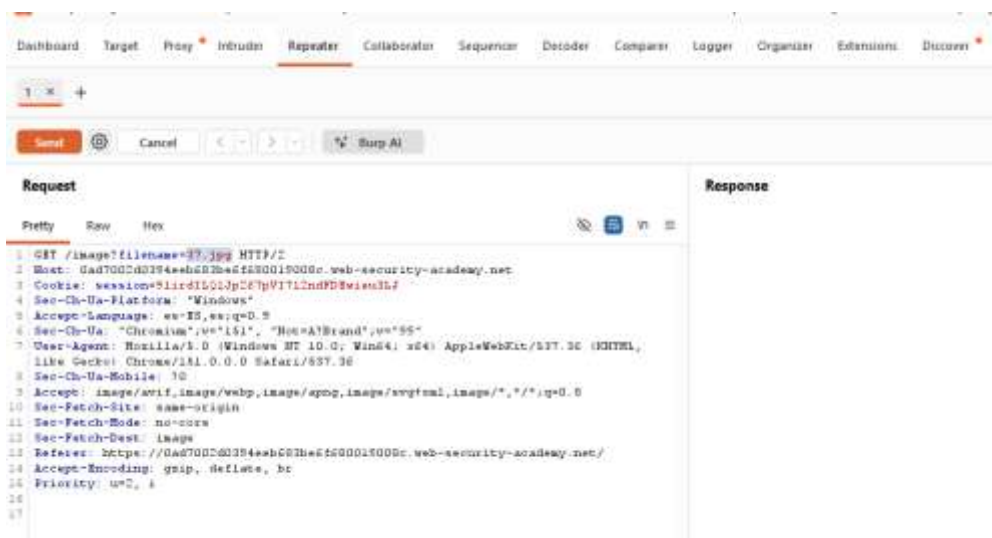


Figura 4. Petición original en Repeater: GET /image?filename=37.jpg.

Un intento inicial de reutilizar el payload clásico de retroceso de directorios (../../../../../etc/passwd) resultaría bloqueado por el filtro de la aplicación, ya que esta elimina activamente las secuencias “../” del valor recibido antes de resolverlo contra el directorio base de imágenes. Esto obliga a plantear una evasión distinta.

6. Fase 3: Explotación — bypass mediante ruta absoluta

Se modificó el valor del parámetro filename en Repeater, reemplazándolo, esta vez, por una ruta absoluta directa, sin ninguna secuencia de retroceso de directorio:

```
GET /image?filename=/etc/passwd HTTP/2
```

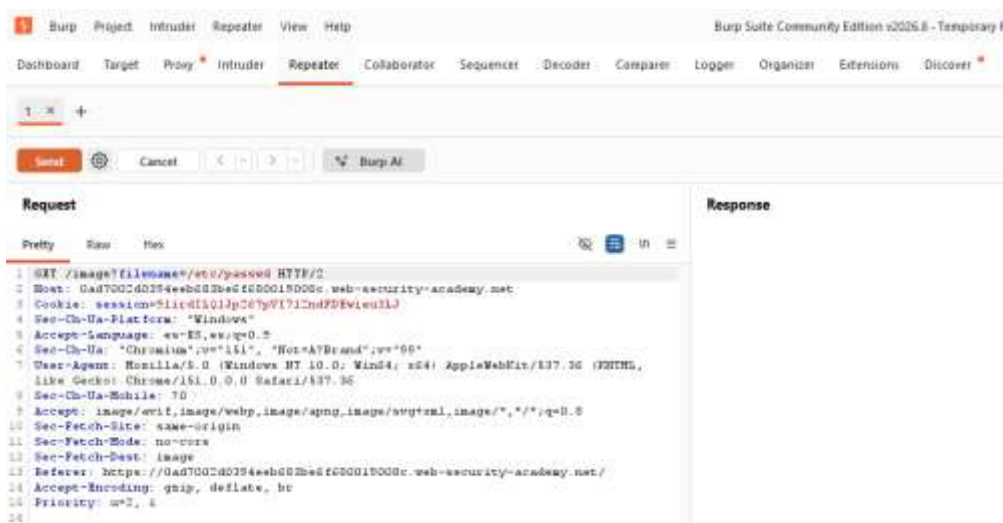


Figura 5. Petición modificada en Repeater con el payload de ruta absoluta /etc/passwd.

Al enviar la petición modificada, el servidor respondió nuevamente con un código HTTP 200 OK y, en el cuerpo de la respuesta, con el contenido íntegro del archivo `/etc/passwd`, confirmando que el filtro de secuencias de traversal fue evadido exitosamente.

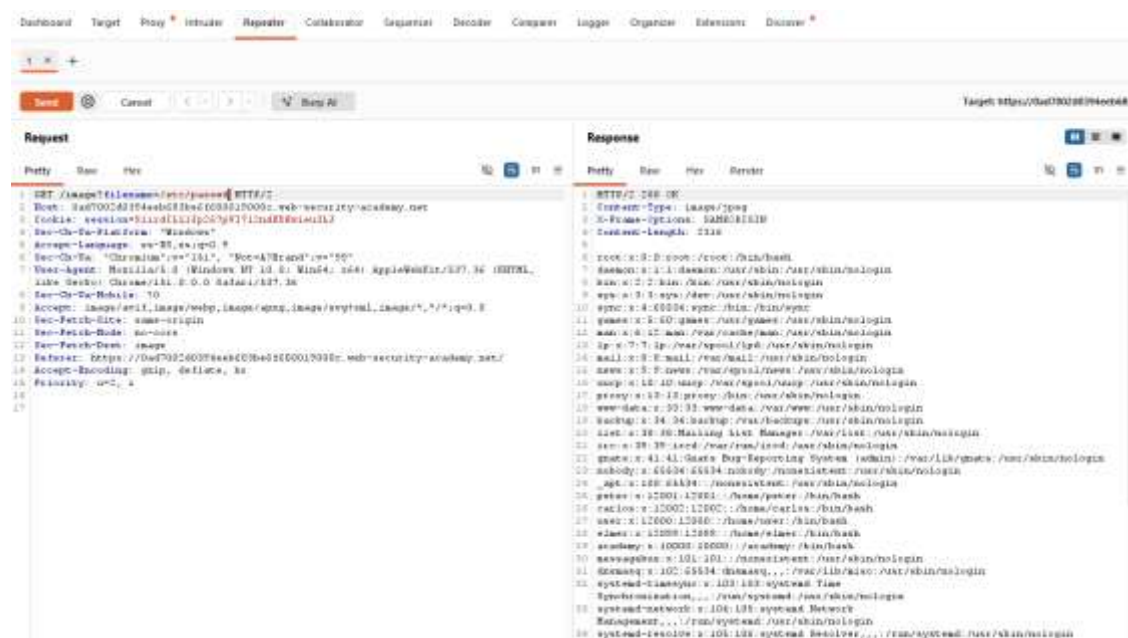


Figura 6. Respuesta del servidor mostrando el contenido completo de `/etc/passwd` tras enviar `filename=/etc/passwd`.

6.1 Explicación del payload

El payload utilizado en este caso es considerablemente más simple que el del laboratorio anterior, y su efectividad radica precisamente en lo que omite:

- `/etc/passwd`: se envía como una ruta absoluta completa, es decir, ya referenciada desde la raíz del sistema de archivos, sin ningún prefijo de retroceso de directorio (`../`).
- Ausencia de secuencias de traversal: al no contener ninguna subcadena “`../`”, el payload atraviesa sin problema el filtro de sanitización de la aplicación, que únicamente está diseñado para detectar y eliminar ese patrón específico.

La causa raíz de la vulnerabilidad es en cómo el servidor construye la ruta final del archivo. En muchos lenguajes y frameworks, al concatenar o unir una ruta base (por ejemplo, `/var/www/images/`) con una ruta proporcionada por el usuario que ya es absoluta (por ejemplo, `/etc/passwd`), la función de manejo de rutas del sistema operativo descarta la ruta base y devuelve únicamente la ruta absoluta recibida. Es decir, aunque la aplicación pretenda anclar la búsqueda dentro del directorio de imágenes, el propio comportamiento de la función de resolución de rutas provoca que el resultado final sea `/etc/passwd`, ignorando por completo el directorio de trabajo por defecto.

Concepto teórico: bypass mediante ruta absoluta (absolute path bypass). Esta técnica de evasión aprovecha una discrepancia entre la defensa implementada (un filtro que busca y elimina secuencias de retroceso de directorio) y el comportamiento real de las funciones de manejo de rutas del sistema operativo o del framework. Dichas funciones, al recibir una ruta absoluta como segundo argumento de una operación de unión de rutas, suelen ignorar la ruta base y devolver

la ruta absoluta tal cual. El resultado es que, aunque la aplicación crea estar restringiendo el acceso a un directorio específico, en realidad basta con solicitar directamente la ruta absoluta del archivo deseado para evadir por completo dicha restricción, sin necesidad de utilizar secuencias “../”.

6.2 Resolución del laboratorio

Al confirmar que la respuesta del servidor contenía el archivo /etc/passwd, PortSwigger marcó automáticamente el laboratorio como resuelto, validando el cumplimiento del objetivo planteado.



Figura 7. Confirmación de PortSwigger: “Congratulations, you solved the lab!”

7. Mitigación recomendada

Además de las medidas generales aplicables a cualquier vulnerabilidad de path traversal, este caso particular exige contramedidas específicas frente al bypass mediante ruta absoluta:

- No basarse únicamente en la eliminación de secuencias “../” como mecanismo de defensa; esta validación es insuficiente por sí sola, como demuestra este laboratorio.
- Rechazar explícitamente cualquier valor de entrada que comience con una barra (/) o con una letra de unidad seguida de dos puntos (por ejemplo, C:), es decir, cualquier indicio de ruta absoluta, además de las secuencias de retroceso.
- Tras sanitizar la entrada, resolver la ruta resultante a su forma canónica (ruta absoluta real, sin enlaces simbólicos ni referencias relativas) y verificar explícitamente que dicha ruta canónica se encuentre dentro del directorio base permitido, antes de acceder al archivo.
- Evitar recibir nombres o rutas de archivo directamente del usuario; sustituir el parámetro filename por un identificador indirecto que el servidor resuelva internamente contra una lista o base de datos de archivos permitidos.
- Ejecutar el servicio con el mínimo privilegio necesario y aislar el proceso mediante chroot jail o contenedores, de forma que, aunque ocurra un bypass, no existan archivos sensibles accesibles.

8. Conclusiones

Este laboratorio demostró que una defensa parcial contra path traversal, centrada únicamente en eliminar secuencias “../”, puede resultar insuficiente cuando la aplicación no valida también la posibilidad de recibir una ruta absoluta. La vulnerabilidad de fondo no desapareció con el filtro implementado; simplemente cambió la forma en que debía explotarse.

El ejercicio reforzó la importancia de comprender el comportamiento interno de las funciones de manejo de rutas del lenguaje o framework utilizado, ya que una defensa que parece razonable a nivel de patrón de texto (buscar “../”) puede ser completamente evadida si no se considera cómo el sistema operativo resuelve las rutas absolutas en operaciones de concatenación o unión de directorios. Esto subraya que las mitigaciones de path traversal deben combinarse con validación de ruta canónica y no depender de un único mecanismo de filtrado.